

Orizon Agents — Week 3 · Technical Documentation & Demo Evidence

What shipped in Week 3 on Stellar testnet, how the money path is kept safe, how to verify it live — and, plainly, what is not evidenced yet.

Programme	Stellar Instawards (Cohort 2026) — Blue Belt Instaward Sprint
Milestone	M3 · Week 3 — Dispute Window & Partial-Credit Refund (Deliverable D3 , Epic 4)
Sprint week	Mon 2026-09-21 → Fri 2026-09-25 · evidence assembled 2026-09-26
Network	Stellar testnet only
Team	Danielle Bagaforo Meer — lead engineer (GitHub ALGOREX-PH) Rieselle Saure ("Rie") — PM + QA (GitHub rie-hash14)
This week	14 pull requests merged · 1,295 commits · +45,936 / -1,152 on main · 908 commits authored by the lead engineer, 268 by QA
Live application	https://orizons.xyz · API https://orizon-agents-be-stellar.onrender.com/docs

CONTENTS

Links at a glance — app, API, repositories, design records, contracts, transactions, PRs	p. 2-3
Walkthrough A — The dispute path, as the buyer sees it	p. 4
A1 The settlement, and the window that opens on it	p. 4
A2 Raising a dispute on one step, with a wallet signature	p. 5
A3 An open dispute: a promise, not a payment	p. 6
A4 A credited dispute, with both artifacts confirmed on-chain	p. 6
A5 A credit recorded whose transfer has not confirmed	p. 7
A6 A rejection, with the platform's reason	p. 7
Walkthrough B — The money path: six things that stop it paying twice, or paying at all	p. 8
B1 The claim row is written before the signature	p. 8
B2 The precondition is a clause of the statement that writes	p. 8
B3 The three-way clamp, and a ceiling that binds while it is finite	p. 9
B4 The adjudication door fails closed, and the switch ships off	p. 10
B5 Nothing reads as done until the chain confirms it	p. 10
B6 The window is stamped on the settlement, never recomputed	p. 11
Walkthrough C — Verify it yourself: the live API, and two transactions on-chain	p. 12
C1 GET /api/stellar/network — testnet, the contracts, and an open decision	p. 12
C2 GET /readiness — our signer is the ledger's authorised scorer	p. 12
C3 GET /openapi.json — six dispute routes, and no operator security scheme	p. 13
C4 The six routes in the interactive docs	p. 13
C5 The refund switch, answered live	p. 14
C6 How independent QA presents the two transactions	p. 14
C7 The refund transfer on Stellar Expert	p. 15
C8 The kind="dispute" rating on Stellar Expert	p. 16
Walkthrough D — Independent QA: what a second pair of eyes found	p. 17
D1 The public UAT repository: suites, drills and evidence	p. 17
D2 The defect register, and the two defects that are not public	p. 18
D3 Eleven found against our own hardening build — including one our fix caused	p. 19
D4 The verdict: no-go	p. 20
Why D3 has no evidence run yet — the gating defect	p. 21
The path to the D3 evidence run, in order	p. 22
Demo video — there is no Week-3 recording; the Week-2 post	p. 23

How to read this. Each step names the page to open, top right, and what to look for. Numbered orange outlines **1** match the numbers in the text; they are drawn by this document over the unaltered screenshot. Every URL here is a live link. **Two kinds of screenshot, never mixed:** Walkthrough A's six frames are local captures against the end-to-end suite's **test fixtures** — every hash in them exists on no ledger — and every page and caption carrying one says so. Everything else was captured on 2026-09-26 from the live deployment and from public GitHub, Stellar Expert and X pages; no wallet was connected and nothing was signed, paid or submitted for any of them.

The live application, the API, the repositories and the design records

Everything below is public. The application and the API run on Stellar testnet; the documentation lives in the public GitHub repositories. Nothing here needs a login.

DEPLOYED APPLICATION AND BACKEND API (LIVE)

Orizon Agents · the live dApp	https://orizons.xyz
Trace / receipt view · where a dispute is raised · A	https://orizons.xyz/app/trace
Interactive API docs · the six dispute routes · C4	https://orizon-agents-be-stellar.onrender.com/docs
Network and contracts · testnet, contract ids, asset · C1	https://orizon-agents-be-stellar.onrender.com/api/stellar/network
Readiness · ratings writer · C2	https://orizon-agents-be-stellar.onrender.com/readiness
OpenAPI document · routes and security schemes · C3	https://orizon-agents-be-stellar.onrender.com/openapi.json

THE FIVE PUBLIC REPOSITORIES

Frontend · the dApp (Next.js)	https://github.com/Bl0cksmiths/Orizon-Agents-FE-Stellar
Backend · FastAPI + Soroban integration	https://github.com/Bl0cksmiths/Orizon-Agents-BE-Stellar
UAT suite · independent QA · D	https://github.com/Bl0cksmiths/Orizon-Agents-UAT-Stellar
Smart contracts · Soroban contracts — no change this week	https://github.com/Bl0cksmiths/Orizon-Agents-Smart-Contract-Stellar
Reference agent · for external operators — no change this week	https://github.com/Bl0cksmiths/Orizon-Agents-Example-Agent-Stellar

DESIGN RECORDS AND THE OPERATOR RUNBOOK FOR THIS EPIC

All under github.com/Bl0cksmiths/Orizon-Agents-BE-Stellar/blob/main/docs/; each row links the full address.

Operator runbook · disputes end to end · B	.../docs/disputes.md
ADR 0002 · partial-credit refund mechanism	.../decisions/0002-partial-credit-refund.md
ADR 0007 · the dispute window and the proof of payer	.../decisions/0007-dispute-window.md
ADR 0008 · who may order a payout, and paying exactly once	.../decisions/0008-refund-execution.md
ADR 0009 · the dispute rating and its replay guard	.../decisions/0009-dispute-rating.md

Contracts, transactions, pull requests, QA — and where the demo video is not

Contracts and transactions open on Stellar Expert (testnet). Each link shows the full id or hash, so the page also works printed.

STELLAR EXPERT — DEPLOYED CONTRACTS (TESTNET, UNCHANGED THIS WEEK)

AgentRegistry · agents, owners, delisting	CAPHXWU53UZUZJGV7IAE57NNMH3YYB5MTW06YA53KKMXSFVL0ITBJ3GQ
ReputationLedger · ratings, including the dispute rating	CDCS0BEVZUPQZV5GV4D6KYHZCLNGW2KXY74RUHSZ3EZUXF34DPW422ZT
PaymentEscrow · x402 workflow caps and the charge	CBJPTMAPM60DGZCZ2IMEQSRUX3WGUXNMKDTNN2KMJ3NFGYZ50J5525PI
AttestationRegistry · sealed run attestations	CBYUZK0ET43UXTBXZUJIBBJW50DGD2J2AZVVXCR3QNGOCAH0XQQHEGK
Asset SAC — native XLM · the configured asset, labelled USDC	CDLZFC3SYJYDZT7K67VZ75HPJVIEUVNIXF47ZG2FB2RMQVU2HHGCYSC

THE TWO TESTNET TRANSACTIONS IN THIS DOCUMENT

Both are **real and confirmed**, and neither is Deliverable 3: they were produced by driving the real backend code against a **drill** ReputationLedger with a test asset, from a developer machine. Walkthrough C sets out exactly what they do and do not prove.

Artifact	Ledger	Transaction (opens on Stellar Expert)
Refund transfer drill asset UATUSD · C7	4865810	a5baac432b582787df0a632b3bc12916c51e12575a8f77bf267fd45b728701b8
kind="dispute" rating drill ReputationLedger · C8	4865811	7138e4e36e47f4f4404b2212aad5584d2f8fb941b387da76acae4c3b4cc07184

MERGED THIS WEEK — 14 PULL REQUESTS, 1,295 COMMITS, +45,936 / -1,152 ON MAIN

Behind them: backend **2,177 tests at 92.53% line coverage** (the gate the 4.07 merge landed under, floor 82%) and frontend **1,503 unit and 149 end-to-end tests**, all passing, CI green on `main` in both. The frontend's end-to-end tests run against a mocked backend with hashes that exist on no ledger — **test evidence, not on-chain evidence**, and the source of Walkthrough A's frames.

BE #60	4.02 — the dispute window, the dispute routes, and a durable settlement record	2026-09-21
BE #61	4.03 — partial-credit refund, review PR (stacked; did not reach main)	2026-09-21
BE #62	4.03 — the same work, landed on main	2026-09-21
BE #63	4.04 — negative on-chain rating for an upheld dispute	2026-09-21
BE #64	4.05 — settlement view for the dispute receipt	2026-09-21
BE #65	4.06 — what a dispute receipt needs to be truthful	2026-09-22
BE #42	chore — untrack evidence and ops docs	2026-09-22
BE #75	4.07 — Epic 4 hardening: money path, disclosure, settlement ratings	2026-09-25
FE #65	Week-2 evidence — Proof of Deliverables summary pages	2026-09-21
FE #66	Week-2 evidence — Technical Documentation PDF, 17 pages	2026-09-21
FE #68	4.05 — dispute action on the trace / receipt view	2026-09-21
FE #69	4.06 — dispute status and refund receipt display	2026-09-22
FE #76	4.07 — the receipt stops stating what it cannot show	2026-09-25
UAT #4	Rie's Week-3 QA — seven 6.03 sub-suites, five drills, D-050 → D-076	2026-09-26

INDEPENDENT QA, AND THE DEMO VIDEO

UAT repository · Playwright suites, five drills, evidence · D	https://github.com/BL0cksmiths/Orizon-Agents-UAT-Stellar
Defect register · 27 defects, D-050 → D-076 · D2	.../docs/uat/defects.md
QA verdict on the dispute story · no-go · D4	.../evidence/6.03-dispute-refund-rating.md
Demo video · there is no Week-3 video — see page 23	the Week-2 build post, labelled as Week 2's

The dispute path, as the buyer sees it

When a paid workflow settles, its buyer has **24 hours** to dispute any single step of it. Proof is a signature from the wallet that paid — no account and no password. An upheld dispute credits part of that step's charge back **from the platform's own wallet**, and writes a permanent negative rating against the agent on the ReputationLedger. Six frames follow the whole path, including the one state this epic exists to get right.

These six frames are local runs against test fixtures, not the deployment. They were captured on 2026-09-26 from the shipped interface at frontend commit `5105a8b`, served by `next dev` and answered entirely by the end-to-end suite's stubs (`e2e/mocks.ts`). **Every transaction hash in them is a fixture: it exists on no ledger, and the Stellar Expert link beside it resolves to nothing.** No wallet was connected to anything real; nothing was signed, paid or submitted. They are evidence of the *interface* — never of a settlement, a refund or a rating having happened. The reason the deployment cannot show this path is on page 21.

A1 The settlement, and the window that opens on it

orizons.xyz/app/trace ➤

Above the trace, a receipt panel states what was charged (0.063 USDC), who paid it, when it settled, and the charge and seal transactions. **DISPUTE WINDOW OPEN · 22h 59m left** carries the closing instant beside it — read from the settlement record, never recomputed (B6).

The terms are stated before any action: an upheld dispute credits **50%** of the step's charge in this fixture, **paid by the platform and never clawed back from the agent**, and the platform decides, with no on-chain arbitration. Each delivered step then carries its own price, a **DISPUTE** button and what it would credit; the step that failed is marked **NOT CHARGED** — “Nothing was charged for this step, so there is nothing to dispute”.

The 50% here is the fixture's policy (`credited_fraction: 0.5`), chosen so a figure on screen can only have been served by the policy and never hard-coded in the copy. The backend's own shipped default is a *full* credit of the step.

The screenshot displays a receipt panel with the following details:

- Receipt** (Status: SETTLED)
- Settled 1h ago · 3 steps
- TOTAL CHARGED**: 0.063 USDC
- PAYER**: GBRPYHIL2CI3FNQ4BXLFMNDLFJUNPU2HY3ZMFHSHONUCEOASW7QC70X2H (Link: VIEW PAYER ON STELLAR.EXPERT)
- SETTLED**: Sep 26, 2026, 6:38 PM GMT+8
- CHARGE**: a41c7e0d93b25f6817ce4a0b9d3f72e15c86a0d4b7e2f91c3a58d06e4b1f7c29 (Link: VIEW CHARGE ON STELLAR.EXPERT)
- SEAL**: 0e9d4c71b3a85f2e6c1d07b94a3e8f52d6c10a7e9b4f38d2c5a16e0b7d93f4a8 (Link: VIEW SEAL ON STELLAR.EXPERT)

Below the receipt, a **DISPUTE WINDOW OPEN** banner indicates 22h 59m left, closing on Sep 27, 2026, 6:38 PM GMT+8.

TERMS: An upheld dispute credits 50% of that step's charge back to you, paid by the platform and never clawed back from the agent. The platform decides each dispute; there is no on-chain arbitration.

The **STEPS** section lists three items:

- 01 seo.brief** (0.009 USDC): outline with 12 target keywords. **DISPUTE** button. Credits 0.0045 USDC if upheld.
- 02 code.gen** (0.054 USDC): calculator app, 3 files. **DISPUTE** button. Credits 0.027 USDC if upheld.
- 03 vision.ocr** (0.012 USDC): **NOT CHARGED**. Nothing was charged for this step, so there is nothing to dispute.

the receipt panel of the trace view (fixture data) · local capture against test fixtures — not the deployment ·
../screenshots/local-01-dispute-action-open-window.png · captured 2026-09-26 19:42 PHT

These six frames are local runs against test fixtures, not the deployment. They were captured on 2026-09-26 from the shipped interface at frontend commit `5105a8b`, served by `next dev` and answered entirely by the end-to-end suite's stubs (`e2e/mocks.ts`). **Every transaction hash in them is a fixture: it exists on no ledger, and the Stellar Expert link beside it resolves to nothing.** No wallet was connected to anything real; nothing was signed, paid or submitted. They are evidence of the *interface* — never of a settlement, a refund or a rating having happened. The reason the deployment cannot show this path is on page 21.

A2 Raising a dispute on one step, with a wallet signature

orizons.xyz/app/trace

Dispute step 2 names the step, what was charged for it and what would be credited if the dispute is upheld, then repeats the three terms, then takes a **required written reason** (500 characters, counted — though the backend still accepts one made only of invisible characters, QA's D-059). **SIGN AND SUBMIT** asks the wallet that paid to sign the message `orizon-dispute:v1:{job_id}:{step}:{nonce}` — domain-separated and covering the exact step, so a captured signature cannot be replayed against another step or another workflow. Signing costs nothing and sends no transaction.

Only the payer is offered this. A connected wallet that is not the payer sees no button and no *disabled* button — no action of any kind. **Nothing was signed for this frame:** the form was filled to show the enabled control, then closed.

DISPUTE

×

Dispute step 2

Ask the platform to credit part of what this step cost.

▶ THE STEP

Step 2 · **code.gen**

calculator app, 3 files

▶ WHAT IS AT STAKE

CHARGED FOR THIS STEP	0.054 USDC
CREDITED IF UPHELD	0.027 USDC

▶ THE TERMS

- An upheld dispute credits 50% of this step's charge back to the wallet that paid.
- The platform pays the credit. Nothing is clawed back from the agent.
- The platform reviews the dispute and decides. There is no on-chain arbitration.

▶ YOUR REASON

Required. Say what went wrong with this step — the platform reads this when it decides.

the calculator app does not compute anything

CANCEL

SIGN AND SUBMIT

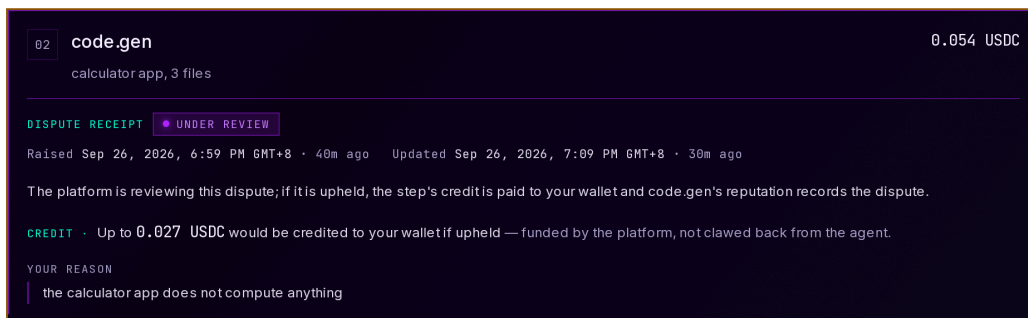
the dispute dialog (fixture data) · local capture against test fixtures — not the deployment ·
../screenshots/local-02-dispute-dialog.png · captured 2026-09-26 19:39 PHT

These six frames are local runs against test fixtures, not the deployment. They were captured on 2026-09-26 from the shipped interface at frontend commit `5105a8b`, served by `next dev` and answered entirely by the end-to-end suite's stubs (`e2e/mocks.ts`). **Every transaction hash in them is a fixture: it exists on no ledger, and the Stellar Expert link beside it resolves to nothing.** No wallet was connected to anything real; nothing was signed, paid or submitted. They are evidence of the *interface* — never of a settlement, a refund or a rating having happened. The reason the deployment cannot show this path is on page 21.

A3 An open dispute: a promise, not a payment

orizons.xyz/app/trace

The badge reads **UNDER REVIEW**, with when it was raised and when it last changed. The credit is stated as a promise and nothing more — “Up to 0.027 USDC **would** be credited ... funded by the platform, not clawed back from the agent” — and the buyer's own reason is shown back to them — in a tab holding the task's read token; without it the payer gets an empty “Your reason” heading (QA's D-067, D-068). **No transaction hash appears in this frame, because nothing has been paid.**



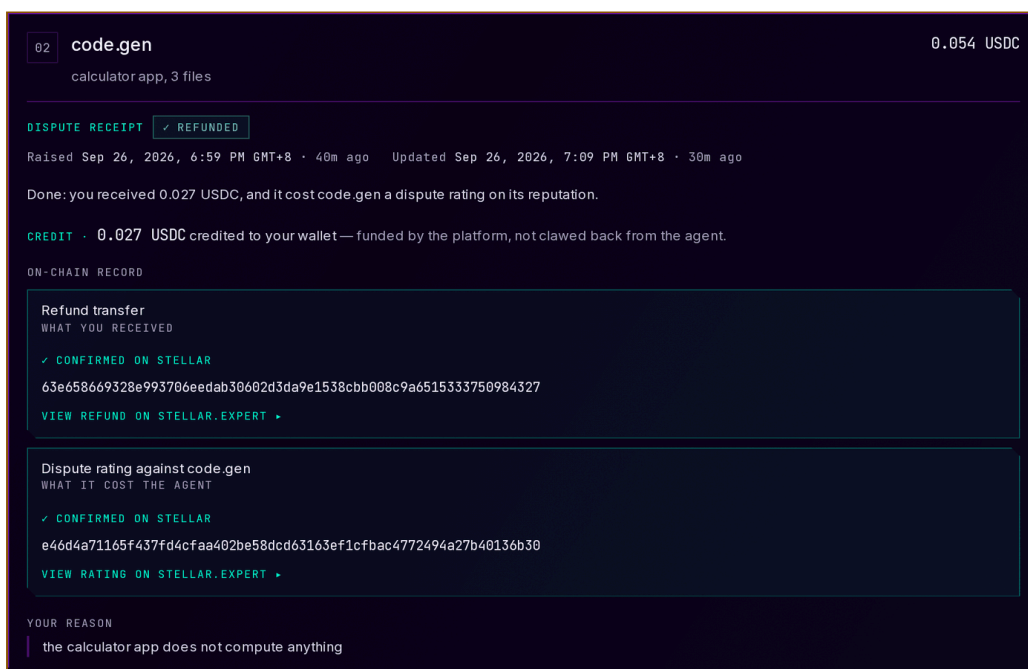
one step's dispute receipt (fixture data) · **local capture against test fixtures — not the deployment** · ../screenshots/local-03-receipt-open-dispute.png · captured 2026-09-26 19:41 PHT

A4 A credited dispute, with both artifacts confirmed on-chain

orizons.xyz/app/trace

REFUNDED, and the sentence says what it cost each side: “you received 0.027 USDC, and it cost code.gen a dispute rating on its reputation”. Below it the two on-chain rows — **Refund transfer** and **Dispute rating against code.gen** — each **CONFIRMED ON STELLAR**, each with its hash painted in full and a link to Stellar Expert.

Hashes are printed whole, in monospace, wrapping: in a screen recording a link cannot be inspected, so the characters on screen are the only thing a reviewer can match against the explorer. **Both hashes here are fixtures and resolve to nothing.** Walkthrough C carries two real ones.



one step's dispute receipt (fixture data) · **local capture against test fixtures — not the deployment** · ../screenshots/local-04-receipt-credited.png · captured 2026-09-26 19:41 PHT

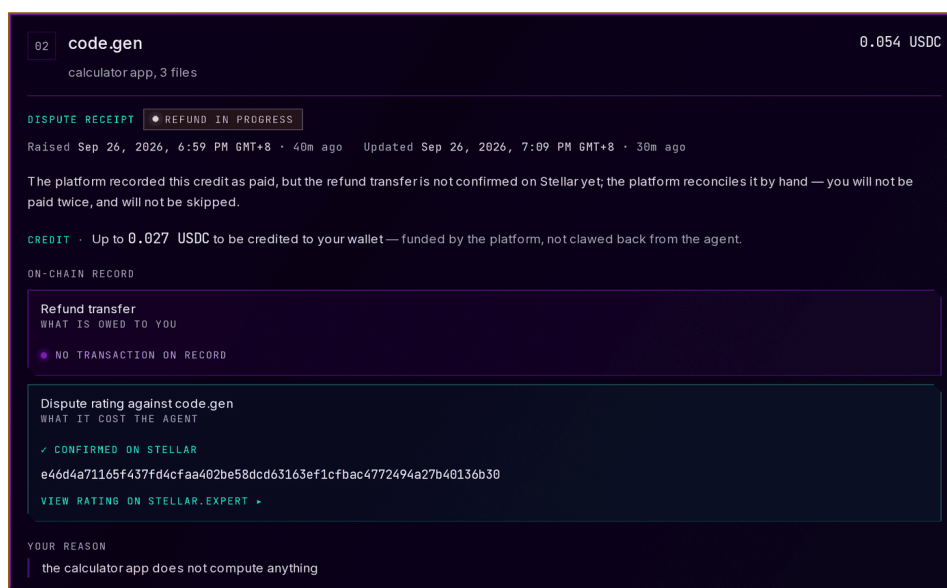
These six frames are local runs against test fixtures, not the deployment. They were captured on 2026-09-26 from the shipped interface at frontend commit `5105a8b`, served by `next dev` and answered entirely by the end-to-end suite's stubs (`e2e/mocks.ts`). **Every transaction hash in them is a fixture: it exists on no ledger, and the Stellar Expert link beside it resolves to nothing.** No wallet was connected to anything real; nothing was signed, paid or submitted. They are evidence of the *interface* — never of a settlement, a refund or a rating having happened. The reason the deployment cannot show this path is on page 21.

A5 A credit recorded whose transfer has not confirmed

orizons.xyz/app/trace

This is the state the epic exists to get right. The platform has recorded the credit, but the chain has not vouched for the transfer. The badge reads **REFUND IN PROGRESS** and never “Refunded”; the credit line stays a promise (“Up to 0.027 USDC to be credited”); and the **Refund transfer** row reads **NO TRANSACTION ON RECORD**, with no hash and no link, beside a dispute-rating row that *is* confirmed.

The sentence is explicit about what happens next: “the platform reconciles it by hand — you will not be paid twice, and will not be skipped.” That is the product face of B5: a timed-out transfer is never retried automatically, because the asset contract has no undo. Paying late is the deliberate trade against ever paying twice.



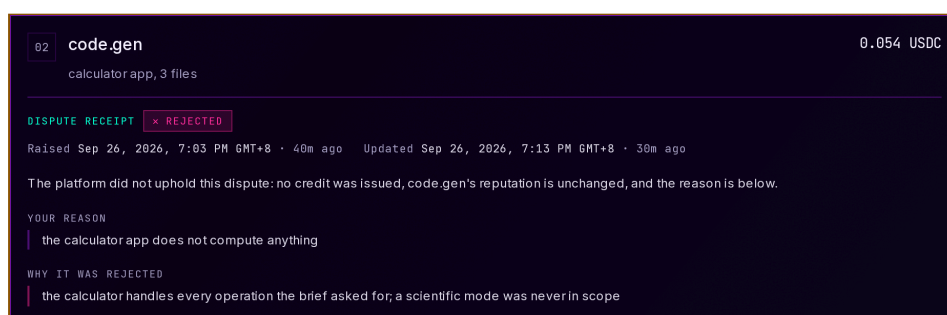
one step's dispute receipt (fixture data) · **local capture against test fixtures — not the deployment** ·
../screenshots/local-05-receipt-crediting-unconfirmed.png · captured 2026-09-26 19:41 PHT

A6 A rejection, with the platform's reason

orizons.xyz/app/trace

REJECTED: “no credit was issued, code.gen's reputation is unchanged, and the reason is below”. The buyer's own reason is shown, and beneath it **WHY IT WAS REJECTED** carries the adjudicator's own words. The note is required on the reject route (1–500 characters), because a rejection with no explanation is worse than no dispute system at all. The backend returns both texts only to a caller holding the task's read token or the operator key, as this frame's tab does; a payer who comes back without the token sees “Rejected” and no reason (QA's D-067). Opening a dispute proves nothing and costs the agent nothing until it is upheld.

No hash and no link appear in this frame, because nothing was paid. Independent QA found a real cost to the disclosure rule that protects this text — see D3 on page 18.



one step's dispute receipt (fixture data) · **local capture against test fixtures — not the deployment** ·
../screenshots/local-06-receipt-rejected.png · captured 2026-09-26 19:43 PHT

The money path: six things that stop it paying twice, or paying at all

An upheld dispute is the only operation in this service that spends the **platform's own balance** on a person's say-so, with nothing on-chain to bound it. A server-signed charge can only ever spend an allowance the payer already authorised; a credit cannot. Everything below exists because of that asymmetry, and every claim names the file and the design record a reviewer can check it against.

B1 The claim row is written before the signature

[github.com](#) · [ADR 0008, decision D2](#) ↗

Upholding happens in a fixed order. The dispute is recorded `upheld`; then a **refund claim** is taken on it — a row in a table, keyed by the dispute id — in the Postgres store it survives a restart — and only one caller can ever hold it; the dispute moves to `crediting`. **If the claim cannot be taken, nothing is signed at all.** Only then is the amount computed, checked against the ceiling, and the transfer signed.

Taking the claim and moving the dispute to `crediting` are **one statement**, not two, and concurrency is settled by the table's primary key rather than by the status the statement read: the loser's `ON CONFLICT DO NOTHING` returns nothing, and nothing returned means sign nothing. With the Postgres store, two adjudicators clicking together, a retried request, a process redeployed mid-flight — all meet the same row. The claim is per dispute; QA's D-058 is the in-memory store's path to a second dispute on one step. A repeat uphold of a `credited` dispute signs no transfer and returns the same refund hash.

D2 — Idempotency is a claim taken before anything is signed

`refund_claims` is one row per dispute that is mid-payment:

```
CREATE TABLE IF NOT EXISTS refund_claims (  
  dispute_id TEXT PRIMARY KEY,  
  claimed_at DOUBLE PRECISION NOT NULL  
);
```

and the primary key is the whole mechanism. `INSERT ... ON CONFLICT (dispute_id) DO NOTHING` is atomic, so of any number of concurrent claimants exactly one inserts and the rest come back empty. `DO NOTHING` rather than `DO UPDATE` keeps the losers on the ordinary empty-result path instead of an exception class every caller would have to name, and leaves the winner's row untouched.

Taking the claim and moving the dispute to `crediting` are one statement, not two, and that is the part worth recording. The version that tried two is worth naming because it is the obvious one: insert the claim, read the status back, append `crediting`. The gap between the insert and the append is a window the process can die in — Render spins a free instance down whenever it idles — and what it leaves behind is a claim row over a dispute still reading `upheld`. Nothing can pay that buyer afterwards: the claim refuses every later claimant, and a release refuses because the dispute is not `crediting`. They would be owed money that no code path could send them. A single statement is its own transaction, so either both rows are there or neither is, whatever happens to the process in between.

Concurrency is settled by the primary key, not by the status the statement reads. Both CTEs share one snapshot, taken before either ran, so two claimants racing each other *both* see `upheld` — a status can rule a claim out and can never arbitrate between two. The unique index is not snapshot-based: exactly one insert lands, the loser's `DO NOTHING` returns nothing, and the outer insert selects through the claim, so the loser writes no event row either. The status gate is still there, on the claim's own `SELECT`, and its job is the other one: a dispute that is not `upheld` inserts nothing at all.

So **None from claim_refund means sign nothing** — already claimed, already credited, still open, or rejected are all the same instruction to the caller. The claim outlives the process that took it, which is the property a lock in memory does not have and the reason it is a table.

Why not a partial unique index over `dispute_events`. ADR 0007 already enforces one dispute per `(job_id_hex, step_index)` with exactly that technique, so reaching for it again is the natural move, and it is wrong for this rule. `dispute_events` is append-only; a uniqueness rule scoped to "is `crediting`" would forbid the **second** claim after a released failure. A buyer whose transfer definitively failed has not been paid, and must stay payable. The duplicate-dispute rule is permanent — a

ADR 0008, decision D2 · <https://github.com/BIOcksmiths/Orizon-Agents-BE-Stellar/blob/main/docs/decisions/0008-refund-execution.md> · captured 2026-09-26 20:16 PHT

B2 The precondition is a clause of the statement that writes

[github.com](#) · [ADR 0008, decision D2](#) ↗

Every status transition on a dispute is a compare-and-set, and the comparison is not a read followed by a write. In the Postgres store, `append_status(..., expected_status=...)` spells the precondition as the final `WHERE` of the `INSERT ... SELECT` that performs the transition: the write lands only while the dispute still reads the status the decision was computed from, and nothing comes back when it does not. The mutex-releasing `DELETE` in the same statement repeats the same clause, so it cannot fire for a transition that was refused.

`uphold` passes `expected_status="open"`, and `reject` does too. Losing the set is answered `409 adjudication_in_progress` or `409 dispute_not_open` — never absorbed. **Stated precisely:** this is a single-statement compare-and-set in the Postgres store. The in-memory fallback store holds the same contract by an ordinary check, which is safe there for a different reason, and which is one of several reasons `DATABASE_URL` must be set in production (page 22, step 5).

B3 The three-way clamp, and a ceiling that binds while it is finite[github.com · docs/disputes.md](https://github.com/B10cksmiths/Orizon-Agents-BE-Stellar/blob/main/docs/disputes.md) ↗

What is transferred is the **smallest of three numbers**: the amount frozen on the dispute when it was opened, so the buyer is never paid less than they were shown and never more; the step's price times the policy fraction in force at adjudication, so the stated policy is honoured; and what the workflow's charge *actually moved on-chain*, so the platform never refunds money it did not collect. With the shipped policy they are normally the same number. Each clamp that bites is logged.

Above that sits a ceiling: while `MAX_REFUND_USDC` (shipped at `1.0`) is finite, a credit over it is **refused before any transaction is built**. It is enforced in the one function that produces a refund amount, so no amount exists that has not been through it — and a non-finite amount is refused first, because a comparison cannot refuse a NaN. The ceiling is deliberately separate from the charge cap: one bounds what a *buyer* authorised themselves to spend, the other what the *platform* pays out of its own wallet. The *setting* itself is never validated, so `MAX_REFUND_USDC=nan` or `inf` is accepted at boot and removes the ceiling — independent QA's D-054, open at this build.

What an upheld dispute pays, and who pays it

The credit is the disputed step's settled charge, under a stated policy rather than a case-by-case judgement: `DISPUTE_CREDITED_FRACTION` ships at `1.0`, the whole of what that step cost. The amount is computed when the dispute is opened and frozen on its record, so a later change to the policy cannot rewrite what the buyer was shown, and it is clamped so a credit can never exceed what was actually charged for the step.

What is actually transferred is the **smallest** of three numbers: the amount frozen on the dispute when it was opened, the policy share of that step's price under the fraction in force at adjudication time, and what the workflow's charge actually moved on-chain. With the shipped policy those are normally the same number. They can differ, because the per-step figure starts life as the plan's *quoted* price while the charge's total is what was really submitted, and when they differ the credit follows the smallest — the platform never refunds money it did not collect.

One consequence of that middle term is worth stating rather than discovering. Because the frozen amount is only ever an upper bound, *lowering* `DISPUTE_CREDITED_FRACTION` does reach disputes that are already open, while raising it cannot. The guarantee is one-directional on purpose: a buyer can never be paid more than the figure they were shown, and a tuning change can never retroactively increase what the platform owes on work already done. A credit is also refused outright above `MAX_REFUND_USDC` (shipped at `1.0`), checked before anything is signed; a step on this deployment settles for hundredths of a USDC, so that ceiling only ever catches something that has gone wrong.

Crediting one step is what makes this a *partial* refund: the rest of the workflow — the steps that did deliver — stays paid, and their agents keep their earnings.

The credit is funded by the platform, and never clawed back from the agent. It is a transfer from the settler's own wallet to the buyer over the asset contract — **not** a reversal of the original charge, and **not** a seizure of anything the agent was paid:

- The deployed `PaymentEscrow` has no refund endpoint and never takes custody — `charge` sends USDC from the payer straight to the agent's owner, so there is nothing held anywhere to reverse.
- Nothing in the system can take funds back out of an agent owner's wallet, and nothing tries to. An operator's settled earnings are final.

The full reasoning, the rejected alternatives and the testnet proof that a real credit lands are in `docs/decisions/0002-partial-credit-refund.md`.

the operator runbook, on what an upheld dispute pays · <https://github.com/B10cksmiths/Orizon-Agents-BE-Stellar/blob/main/docs/disputes.md> · captured 2026-09-26 20:05 PHT

B4 The adjudication door fails closed, and the switch ships off[github.com · docs/disputes.md](https://github.com/docs/disputes.md)

`uphold` and `reject` are **the only two routes in the service that fail closed**. Every other route treats an unset `API_KEY` as “the demo is open”; these two treat it as *refuse*, on every network including testnet, and they refuse while `DISPUTE_REFUNDS_ENABLED` is false — **the shipped default**. The key is compared in constant time, and an absent header can never match an unset secret.

Turning the switch on without a key is not a silent weakness: `DISPUTE_REFUNDS_ENABLED=true` alone makes the process **refuse to start** unless `API_KEY` is set, and the error says why. There is no order of setting the two in which the rule does not bite. The switch is checked twice — at the route and again inside the service — because an operator script that imports the service credits a buyer without passing through the web framework at all, and a switch only one door honours is not a switch. The live deployment’s answer today is on page 14.

B5 Nothing reads as done until the chain confirms it[github.com · docs/disputes.md](https://github.com/docs/disputes.md)

A dispute becomes `credited` only on an exact success *with a hash*. A definitive failure releases the claim and returns the dispute to `upheld`, payable again — a buyer who was not paid stays payable. A **timeout is never retried**: the dispute stays in `crediting` with the in-flight hash recorded, for a person to reconcile against the chain. That is A5 on screen.

On the rating side the distinction is a field, not careful wording: `rating_confirmed` is true only once the ledger has vouched for the submission, and a hash recorded for a submission that timed out sets it false. A failed rating never touches the refund — the buyer keeps the credit. One honest asymmetry, and the hardening PR lists it as not-in-this-pass: there is **no matching `refund_confirmed` field**; the interface reads an unconfirmed refund from the absence of a recorded transfer instead, which is the row A5 shows.

For operators: where the records live

Settlements and disputes are the first things this backend keeps that are not on-chain and not disposable. They live in Postgres when `DATABASE_URL` is set, behind the same store seam the endpoint bindings use, in append-only tables: a dispute’s history is written, never rewritten, so its status is the newest row rather than a field that was overwritten.

The refund claims are the one table that is not append-only, because it is not a record — it is the mutex, one row per payout in flight, taken before anything is signed and dropped inside the same statement that credits, releases or rejects. It is also the reconciliation queue, which is the section below.

Without `DATABASE_URL` the service falls back to an in-memory store, and the whole of this document becomes true only until the next restart — which a free-tier instance performs whenever it idles. The fallback is there so local development and the test suite need no database; it is not a deployment. It says so once at startup, and it logs a warning naming any record it drops, so a window that can no longer be honoured is never silent.

One read is weaker than the records behind it. With `TASK_AUTH_REQUIRED` on, `GET /api/tasks/{task_id}/disputes` is gated by the task’s read token, and those tokens live in memory with the task state, not in Postgres — so after a restart that read answers as though the task were unknown, even though the settlement and its disputes survived. A dispute already opened is unaffected: `GET /api/disputes/{dispute_id}` keeps working, and opening one is gated by the payer’s wallet signature, never by the task token. A **first** dispute is not unaffected, because this read is where the console gets the job id to ask for a challenge; after a restart under enforcement, the buyer’s console cannot start one, and an operator can read the same view with the API key. With enforcement off, the default, none of this applies: the read is served from the durable records and survives a restart with them.

A settlement is recorded after the charge and the seal have landed, so it can never fail the workflow. If it cannot be written, the workflow is paid and attested but has **no dispute window**, and the only trace of that is the error log — which is why that failure is logged with the same weight as an attestation that did not settle.

the operator runbook, on where the records live · <https://github.com/BIOcksmiths/Orizon-Agents-BE-Stellar/blob/main/docs/disputes.md> · captured 2026-09-26 20:05 PHT

B6 The window is stamped on the settlement, never recomputed

[github.com](#) · [ADR 0007, decision D1](#) ↗

`SettlementRecord.window_closes_at` is written **once**, at settlement, from the value of `DISPUTE_WINDOW_SECONDS` (shipped at `86400.0`) in force at that instant. Every later check compares against that stored field. Across the whole service there is exactly one place the arithmetic happens, and every other reference reads the field.

That is the difference between a promise and a setting. Recomputed on read, retuning the window would silently move deadlines for work already done — shortening it closes windows a buyer was told were open; lengthening it reopens windows an operator had been told were closed and whose earnings they believed final. The stamp makes the knob mean what an operator expects: it governs what settles *after* the change, and nothing that already happened. A dispute arriving on the exact stamped second is inside the window, and a late dispute is refused with **when** it closed, not merely that it did.

D1 — A 24-hour window, opening when the workflow settles, stamped on the record

`DISPUTE_WINDOW_SECONDS` ships at `86_400.0` — 24 hours — measured from the moment the workflow settles, which is the same moment `_settle_onchain` charges and seals. The window opens at settlement rather than at task completion because settlement is the event that creates something to dispute: before the charge there is no money at stake, and a run that never charged has nothing to credit back.

24 hours is a product choice with two constraints on it, and it sits between them rather than being derived from either. Short enough that an operator's earnings become final within a day and the platform's exposure to a credit is bounded; long enough that a buyer in any timezone gets a full waking day to look at the artifact they paid for. A demo-length window — an hour, say — would make the feature testable and the promise worthless.

The closing time is stamped on the settlement record, not recomputed from config on read. `SettlementRecord.window_closes_at` is written once, at settlement, from the value of `DISPUTE_WINDOW_SECONDS` in force at that instant, and every later check compares against that field.

This is the part worth the ADR. The alternative — store `settled_at` and evaluate `settled_at + settings.dispute_window_seconds` whenever someone asks — is one field lighter and wrong in a way that only shows up in production. A buyer was *told* a closing time: it is on their receipt and in the API response they got when the workflow settled. Retuning the setting afterwards, for any ordinary reason — shortening it to cut platform exposure, lengthening it for a support case — would silently move the deadline for work already done, in both directions. Shortening it closes windows the platform promised were open; lengthening it reopens windows an operator was told had closed and whose earnings they believed final. Neither outcome is one anyone would trace back to the config change that caused it.

The stamp makes the tuning knob mean exactly what an operator expects: it governs workflows that settle *after* the change, and nothing that already happened. The same reasoning is written at both ends — beside the setting in `app/config.py` and on the field in `dispute_store.SettlementRecord` — because it is the kind of "redundant" field a later cleanup deletes.

Timestamps are epoch seconds from our own clock, never the database's, so no timezone conversion sits between what the buyer was promised and what is later read back.

ADR 0007, decision D1 · <https://github.com/BI0cksmiths/Orizon-Agents-BE-Stellar/blob/main/docs/decisions/0007-dispute-window.md> · captured 2026-09-26 20:05 PHT

Where to check each safeguard

Safeguard	Source, on <code>main</code> of the backend repository	Design record
B1 · claim before signature	<code>app/services/dispute_svc.py</code> <code>uphold</code> · <code>app/services/dispute_store.py</code> <code>claim_refund</code>	ADR 0008 D2
B2 · compare-and-set	<code>app/services/dispute_store.py</code> <code>append_status(expected_status=...)</code>	ADR 0008 D2
B3 · clamp and ceiling	<code>app/services/refund_svc.py</code> <code>creditable_for</code> · <code>app/config.py</code>	ADR 0008 D4, D5
B4 · door and switch	<code>app/security.py</code> <code>require_adjudicator</code> · <code>app/config.py</code>	ADR 0008 D1
B5 · the chain confirms	<code>app/services/refund_svc.py</code> · <code>app/services/dispute_svc.py</code> <code>_apply_rating</code>	ADR 0008 D3 · 0009 D3, D4
B6 · the window, stamped	<code>app/services/execution_svc.py</code> <code>_record_settlement</code> · <code>dispute_store.py</code>	ADR 0007 D1, D3

Verify it yourself: the live API, and two transactions on-chain

Nothing in this section needs the dApp, and nothing needs our word for it. The four readings below were taken against the live deployment while this document was assembled; the first request after a quiet period can take 30–60 seconds while the free-tier instance wakes. The JSON boxes are the live response bodies re-printed with indentation — no value is changed.

C1 GET /api/stellar/network — testnet, the contracts, and an open decision

[live response ↗](#)

The network is **testnet**, and the four application contract ids are the ones deployed in Week 2 and listed on page 3 — Epic 4 required no contract change, and the dispute rating is written through the ReputationLedger already deployed then.

The configured asset is not USDC, and the interface says “USDC”. The endpoint reports `asset: "native"`, and `asset_sac` is the **native XLM Stellar Asset Contract on testnet** — derivable from the asset itself, so it can be checked without trusting this document. Every amount in the product — plan prices, charges, credits, the receipt's “credited to your wallet” — is labelled USDC while the deployment moves test XLM. **This is an open decision, not a resolved one:** either a USDC SAC is configured or the label is corrected, and D3 evidence should not be captured while the two disagree (page 22, step 7).

```
GET https://orizon-agents-be-stellar.onrender.com/api/stellar/network
fetched 2026-09-26 20:06 PHT · response body, formatted · tinted lines are the fields the text refers to

{
  "network": "testnet",
  "rpc_url": "https://soroban-testnet.stellar.org",
  "network_passphrase": "Test SDF Network ; September 2015",
  "admin": "GA7AI5TAJEZA27I666DSJC4MUJYBEWUYNZWP7R2ONA7IZQV06R50QV",
  "dispatch_signer": "GB5MKHDFLJZ60FPAHM7R4HGBUPFV5PZYL3W27VTIUZZ25JMQSDZBKCMR",
  "asset": "native",
  "asset_sac": "CDLZFC3SYJDZT7K67VZ75HPJVEUVNIXF47ZG2FB2RMQQVU2HHGCYSC",
  "contracts": {
    "agent_registry": "CAPHXWU53UZUZJGV7IAE57NNMH3YYB5MTW06YA53KKMXSFVLOITBJ3GQ",
    "reputation_ledger": "CDCS0BEVZUPQZV5GV4D6KYHZCLNGW2KXY74RUHSZ3EZUXF34DPW422ZT",
    "payment_escrow": "CBJPTMAPMGDGCZC2IMEQSRUX3WGUXNMKDTNN2KMJ3NFGYZ50J5525PI",
    "attestation_registry": "CBYUZK0ET43UXTBXUJIBBJW50DGD2J2AZVXXCR3Q0NG0CAHOXQQHEGK"
  }
}
```

response body, formatted · <https://orizon-agents-be-stellar.onrender.com/api/stellar/network> · captured 2026-09-26 20:06 PHT

C2 GET /readiness — our signer is the ledger’s authorised scorer

[live response ↗](#)

`ratings.writer` reads **"scorer"**: the deployment's signing key **is** the ReputationLedger's authorised Scorer, so a dispute rating it signs can land. That authorization is itself on-chain, from Week 2 — `ReputationLedger.set_scorer`. **This is the one live precondition for D3 that holds today.**

The signer is `GDB4N25UYM3YNTTAWX7LSGI2P70R62QZQXRNQWAGF5TFVENDKCTTCDHP`. Keep it: C7 and C8 are signed by a different account, and that difference is the whole point of them.

```
GET https://orizon-agents-be-stellar.onrender.com/readiness
fetched 2026-09-26 20:06 PHT · response body, formatted · tinted lines are the fields the text refers to

{
  "status": "ready",
  "llm": "ok",
  "stellar": "configured",
  "signer": "configured",
  "pdax": "configured",
  "cold_start": {
    "routable": true,
    "lower_bound_bps": 5677,
    "floor_bps": 5500,
    "margin_bps": 177
  },
  "ratings": {
    "writer": "scorer",
    "signer": "GDB4N25UYM3YNTTAWX7LSGI2P70R62QZQXRNQWAGF5TFVENDKCTTCDHP",
    "scorer": "GDB4N25UYM3YNTTAWX7LSGI2P70R62QZQXRNQWAGF5TFVENDKCTTCDHP"
  }
}
```

response body, formatted · <https://orizon-agents-be-stellar.onrender.com/readiness> · captured 2026-09-26 20:06 PHT

C3 GET /openapi.json — six dispute routes, and no operator security scheme

[live document ↗](#)

All six dispute routes are served. But the document declares **no security scheme at all**: `components` holds only `schemas`, `components.securitySchemes` is absent, there is no top-level `security`, and **no operation carries one**.

That is a dated fingerprint. The 4.07 hardening merge ([BE #75](#), 2026-09-25) declares the operator key as an OpenAPI security scheme named `OperatorApiKey` on both adjudication routes, and pins it with a test. A live document without it therefore **predates that merge** — so the deployment is behind `main` and does not carry the hardening pass's fixes, including the double-payment fix and the disclosure fix. Render does not auto-deploy for this organisation, so the redeploy is a manual step (page 22, step 2).

GET https://orizon-agents-be-stellar.onrender.com/openapi.json	
fetched 2026-09-26 20:07 PHT · each row is a jq query against the live document and the answer it returns · tinted rows are the fields the te	
<code>.openapi</code>	3.1.0
<code>.info.title + " " + .info.version</code>	Orizon Agents API 0.1.0
<code>.paths length</code>	75 paths, 76 operations
<code>[.paths keys[] select(test("disput"))]</code>	6 dispute routes – listed below
<code>.components keys</code>	["schemas"]
<code>.components.securitySchemes</code>	null (the key is absent)
<code>.security</code>	null (the key is absent)
<code>[.paths[] select(has("security"))] length</code>	0 (no operation declares one)
POST	/api/disputes/challenge
POST	/api/disputes
GET	/api/disputes/{dispute_id}
GET	/api/tasks/{task_id}/disputes
POST	/api/disputes/{dispute_id}/uphold
POST	/api/disputes/{dispute_id}/reject

each row is a query against the live document and the answer it returns · <https://orizon-agents-be-stellar.onrender.com/openapi.json> · captured 2026-09-26 20:07 PHT

C4 The six routes in the interactive docs

[live docs ↗](#)

The same six routes as the deployment serves them, each try-able from the browser: mint a challenge to sign, open a dispute on a settled step, read one dispute, read a task's window and its disputes, and the two adjudication routes — **uphold** and **reject**. Consistent with C3, the page shows no *Authorize* control, because the deployed document declares no scheme for one to fill.

disputes Dispute a settled step inside its 24-hour window, proved by the payer's wallet signature; and the operator-keyed adjudication that upholds one into a credit or rejects it.		⤴
POST	/api/disputes/challenge	Mint a dispute challenge to sign
POST	/api/disputes	Open a dispute on a settled step
GET	/api/disputes/{dispute_id}	Read one dispute
GET	/api/tasks/{task_id}/disputes	A task's dispute window and the disputes raised on it
POST	/api/disputes/{dispute_id}/uphold	Uphold a dispute and credit the buyer
POST	/api/disputes/{dispute_id}/reject	Reject a dispute, with a reason the buyer is shown

the disputes group of the interactive API docs · <https://orizon-agents-be-stellar.onrender.com/docs> · captured 2026-09-26 20:01 PHT

C5 The refund switch, answered live[live response ↗](#)

Posting to the uphold route with no credential, against a dispute id that exists on no deployment, is answered **503 dispute_refunds_disabled**. The route refuses **before** it looks at the adjudicator key and before it looks for the dispute — the same answer a bogus key gets. **Dispute refunds are switched off on the deployment**, which is the correct default for a money path nobody has deliberately turned on, and is QA's D-051 ([backend #68](#)), filed as a configuration state rather than a defect in the code.

Repeat it from a terminal:

```
curl -X POST https://orizon-agents-be-stellar.onrender.com/api/disputes/x/uphold.
```

POST /api/disputes/{dispute_id}/uphold

sent 2026-09-26 20:07 PHT from the browser · no credential, and a dispute id that exists on no deployment

request

```
POST https://orizon-agents-be-stellar.onrender.com/api/disputes/00000000-0000-0000-0000-000000000000/uphold
content-type: application/json
{}

response · 66 ms
```

HTTP 503

```
{
  "detail": "dispute_refunds_disabled",
  "error": {
    "code": "dispute_refunds_disabled",
    "message": "dispute refunds disabled",
    "request_id": "89b0ce3825f3418b"
  }
}
```

the live POST and its response body, formatted · the route in the API docs · <https://orizon-agents-be-stellar.onrender.com/docs> · captured 2026-09-26 20:07 PHT

C6 How independent QA presents the two transactions[github.com](#) · [QA 6.03 evidence, §2 ↗](#)

Because nothing settles and nothing can be upheld on the deployment, the closest real evidence came from QA driving the real backend and the real trace page on her own machine, against a ReputationLedger **she deployed herself** from the same WASM the deployed ledger runs, paying credits in a test asset the drill issues. One uphold produced two real testnet transactions, both re-read on Horizon as `successful: true`. Her framing, unedited, is the shot above; ours does not soften it.

2 · Deliverable 3 evidence

The card asks for three artifacts: the dispute tx, the refund tx and a recording of the dispute UI. None can be captured on the deploy, for the reasons in §1.

The closest real evidence comes from the 6.03f drill. The real backend and the real trace page ran on this machine against the 6.03e drill's own ReputationLedger (testnet) and a test asset, UATUSD. One uphold through `POST /api/disputes/{id}/uphold` produced:

artifact	tx	ledger	resolves
refund transfer	a5baac43...01b8	4865810	yes, re-read on Horizon 2026-09-26, <code>successful: true</code>
kind="dispute" rating	7138e4e3...7184	4865811	yes, re-read on Horizon 2026-09-26, <code>successful: true</code>

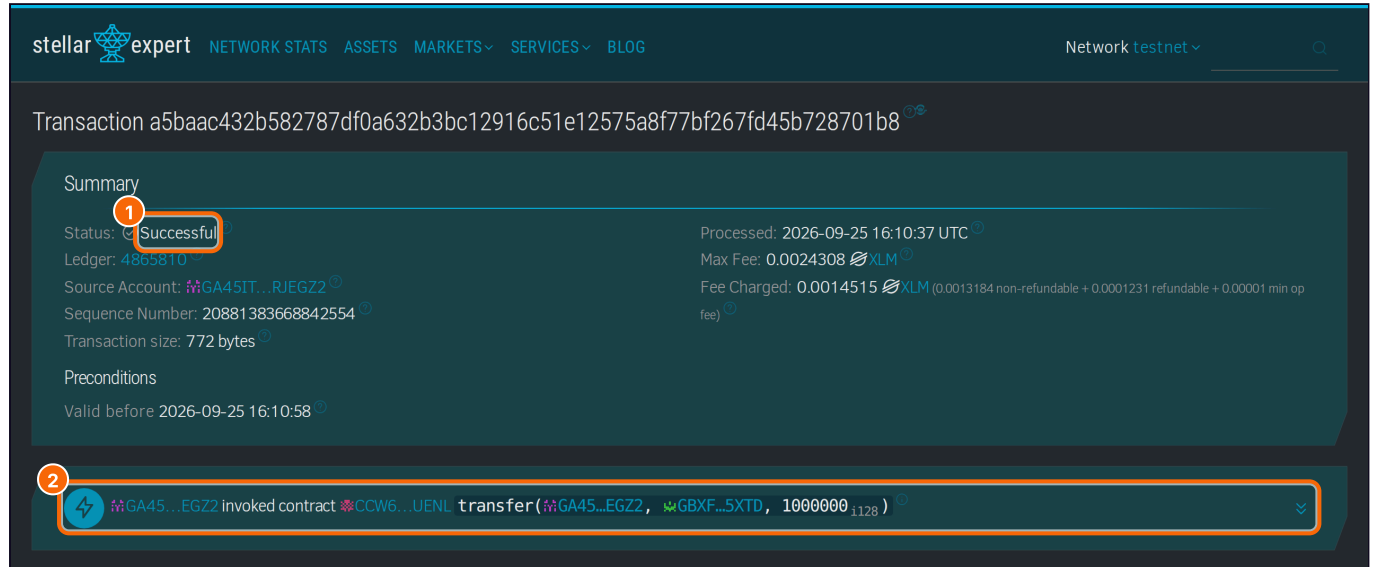
These prove the code path. They are **not** Deliverable 3. The asset is not USDC, the ledger is not the platform's, and the service is not the deploy. Deliverable 3 stays open until one run on the deploy produces both transactions. The recording must be captured in that same session, following `checklists/6.03f-phone-and-screen-reader.md`.

QA's own evidence file, section 2 · <https://github.com/BI0cksmiths/Orizon-Agents-UAT-Stellar/blob/main/docs/uat/evidence/6.03-dispute-refund-rating.md> · captured 2026-09-26 20:06 PHT

C7 The refund transfer on Stellar Expert[stellar.expert](#) · tx a5baac43...

Successful ①, ledger 4865810, 2026-09-25 16:10:37 UTC. The invocation ② is a `transfer` on the drill's asset contract `CCW6...UENL`, from `GA45...EGZ2` to the drill's payer, for **1,000,000 stroops** — 0.1 of the drill's test asset. Re-read on Horizon while this document was assembled: `successful: true`, at that ledger.

Note the source account, and note the asset contract: neither is the deployment's. That is exactly why this is proof of the code path and not of Deliverable 3.



stellar expert NETWORK STATS ASSETS MARKETS SERVICES BLOG Network testnet

Transaction a5baac432b582787df0a632b3bc12916c51e12575a8f77bf267fd45b728701b8

Summary

Status: **Successful** ①

Ledger: 4865810

Source Account: GA45IT...RJEZ2

Sequence Number: 20881383668842554

Transaction size: 772 bytes

Preconditions

Valid before 2026-09-25 16:10:58

Processed: 2026-09-25 16:10:37 UTC

Max Fee: 0.0024308 XLM

Fee Charged: 0.0014515 XLM (0.0013184 non-refundable + 0.0001231 refundable + 0.000001 min op fee)

② GA45...EGZ2 invoked contract CCW6...UENL transfer(GA45...EGZ2, GBXF...5XTD, 1000000 i128)

Stellar Expert (testnet) · <https://stellar.expert/explorer/testnet/tx/a5baac432b582787df0a632b3bc12916c51e12575a8f77bf267fd45b728701b8> · captured 2026-09-26 20:01 PHT

C8 The kind="dispute" rating on Stellar Expert

stellar.expert · tx 7138e4e3...

Successful ①, ledger 4865811 — five seconds after the refund, and only because the refund landed and was recorded first. The invocation ② is a `submit` on the drill's ReputationLedger naming the agent and `dispute`, scored 10 out of 100 and weighted by the step's quoted price.

The rating is written under a **derived** job id, because the ledger's replay guard keys on `(agent_id, job_id)` and the settler has already auto-rated that step at settlement. The derivation keeps the sealed job's own first eight bytes in front, so a reviewer can tie a rating to its job **by eye**, and hashes the step behind them, so two upheld disputes in one job produce two ratings rather than one and a refusal. That replay guard is also the rating's idempotency: retrying a rating is always safe, and retrying an unconfirmed refund never is.

Stellar Expert (testnet) · <https://stellar.expert/explorer/testnet/tx/7138e4e36e47f4f4404b2212aad5584d2f8fb941b387da76acae4c3b4cc07184> · captured 2026-09-26 20:01 PHT

What the two transactions are, and what they are not

They are **real, confirmed testnet transactions**, produced by one uphold through `POST /api/disputes/{id}/uphold` against the real backend code. Independent QA, who produced them, states in her own evidence: *"These prove the code path. They are **not** Deliverable 3. The asset is not USDC, the ledger is not the platform's, and the service is not the deploy."*

Why each is not the deliverable	Checkable on Stellar Expert
The asset is a test asset	the transfer moves <code>UATUSD</code> , issued by the drill, over its own SAC — not the configured asset SAC
The ledger is not the platform's	the rating went to the drill's own ReputationLedger, not <code>CDCS0BEV...22ZT</code> , so it does not appear in the reputation the live planner routes on
The service is not the deployment	the source account on both is <code>GA45ITAK...EGZ2</code> , the drill's settler — not the deployment's signer <code>GDB4N25U...CDHP</code> , which C2 shows live

Independent QA: what a second pair of eyes found

QA is a separate person, a separate repository and a separate verdict. Rie owns stories 6.03a–6.03g, and all of the work is public: the Playwright suites, the drills written to reach behaviour the deployment cannot show, the per-story evidence pages, and a defect register that does not spare our own build. Her verdict on the dispute story is **no-go**, and it stands.

D1

The public UAT repository: suites, drills and evidence

github.com · Orizon-Agents-UAT-Stellar ↗

`tests/` holds 22 Playwright specs run against the **live deployment** across four browser projects — there is deliberately no local server in the configuration, so a pass there is a pass for a user. Six of them are this epic's: the dispute path, the refusal matrix, eligibility, durability, the reputation consequence and the adjudication door. `docs/uat/` holds the test plan, the traceability matrix, the per-story evidence and the defect register.

`tools/` holds **five drills** written this week, precisely because the deployment cannot hold a dispute: a restart drill that hard-kills a real backend on real PostgreSQL and compares the API either side; a reputation drill that deploys its own ReputationLedger and issues its own test asset; a dispute-UI drill that seeds every receipt state against the real trace page; an adjudication drill that boots the service once per configuration and proves nothing was signed by reading the settler's sequence number off Horizon before and after; and a rating-log drill that needs no network at all. Each carries a README stating it never uses the deployment's signer, ledger or money.

 ci	added e2e ci workflow module	2 weeks ago
 docs	updated uat defects module	5 hours ago
 tests	updated adjudication door spec module	18 hours ago
 tools	updated restart drill browser module	6 hours ago
 .env.uat.example	added uat env example module	2 weeks ago
 .gitignore	updated gitignore module	2 weeks ago
 README.md	updated readme with suite layout module	2 weeks ago
 package-lock.json	added uat lockfile module	2 weeks ago
 package.json	added package manifest module	2 weeks ago
 playwright.config.ts	updated playwright config module	2 weeks ago
 tsconfig.json	updated tsconfig module	last week

the repository's file listing · <https://github.com/BIOcksmiths/Orizon-Agents-UAT-Stellar> · captured 2026-09-26 20:05 PHT

D2 The defect register, and the two defects that are not publicgithub.com · [docs/uat/defects.md](https://docs.uat/defects.md) ↗

Twenty-seven defects were logged this week, D-050 to D-076, each a written reproduction rather than a line in a table. **Twenty-five are filed as public GitHub issues** in the repository that owns the code — sixteen in the backend, nine in the frontend — each quoting the failing criterion and linking back to the register.

Two are deliberately not public. D-053 and D-058 are money-path defects; their mechanisms and reproductions were withheld from the public log and handed to the settler key holder and the backend maintainers directly. D-053 is the one that matters: under one specific interleaving of adjudication calls the refund guard did not hold. Triggering it needs adjudicator credentials, so no buyer or anonymous caller could reach it, and nothing happened on-chain because refunds are off. **QA found it on 2026-09-24, a day before our own audit reproduced and fixed the same defect**, and her standing advice was not to enable refunds until it was fixed. Her re-check against the hardening build produced one transfer where the earlier build signed two.

The table opposite is the register's own mapping, and it is the thing to check rather than our summary of it. Two details a careful reader will notice, stated here rather than glossed: the numbering is **not contiguous** — backend #75 and frontend #76 belong to nobody in this list — and the table's preamble still says "stories 6.05 and 6.06 ... filed 2026-09-24", which is older than several of the rows beneath it. The rows are right; the sentence above them has not caught up.

Bug issues			
Every open defect from stories 6.05 and 6.06 is filed as a Bug in the repository that owns the code, each quoting the failing Given/When/Then and linking back here. Filed 2026-09-24.			
defect	severity	repo	issue
D-042	Critical	Example-Agent	#2
D-047	Major	Example-Agent	#3
D-048	Minor	Example-Agent	#4
D-049	Major	Example-Agent	#5
D-044	Major	FE	#70
D-045	Minor	FE	#71
D-046	Major	FE	#72
D-043	Major	BE	#66
D-050	Critical	BE	#67
D-039	Critical	Smart-Contract	#3
D-051	Blocker (6.03a)	BE	#68
D-052	Minor	BE	#69
D-054	Major	BE	#70
D-055	Minor	BE	#71
D-056	Minor	BE	#72
D-057	Minor	FE	#73
D-059	Minor	BE	#73
D-060	Minor	FE	#74
D-061	Minor	FE	#75
D-062	Minor	BE	#74
D-063	Minor	BE	#76
D-064	Major	BE	#77
D-065	Minor	FE	#77
D-066	Major	BE	#78
D-067	Major	BE	#79
D-068	Minor	FE	#78
D-069	Major	FE	#79
D-070	Minor	FE	#80
D-071	Minor	FE	#81
D-072	Minor	FE	#82
D-073	Minor	BE	#80
D-074	Major	BE	#81
D-075	Minor	BE	#82
D-076	Minor	BE	#83

D-036, D-037, D-038 and D-040 are not filed: they were resolved by the 2026-09-24 redeploy. D-050 is D-039's consequence and says so in both issues.

The shape of the week: 1 Blocker, 2 Critical, 7 Major, 17 Minor

Blocker · 1 D-051 — refunds switched off on the deployment (story 6.03a); "deployment configuration, not code"

Critical · 2 D-050 — no run can be disputed at all · D-053 — the adjudication concurrency defect, held privately

Major · 7 D-054, D-058, D-064, D-066, D-067, D-069, D-074

Minor · 17 the rest — but two of them, D-075 and D-076, were filed as Urgent because they sit on the money path

Three of the 27 are already **fixed in code and not yet deployed** (D-053, D-064, and D-056 for the buyer's routes), so **"27 logged" is the honest count, not "27 open"**. None is marked resolved, because QA counts a defect resolved only once the deployment has taken the build and the criterion passes there.

the register's own defect-to-issue table ·

[https://github.com/BIOcksmiths/Orizon-Agents-UAT-](https://github.com/BIOcksmiths/Orizon-Agents-UAT-Stellar/blob/main/docs/uat/defects.md)

[Stellar/blob/main/docs/uat/defects.md](https://github.com/BIOcksmiths/Orizon-Agents-UAT-Stellar/blob/main/docs/uat/defects.md) · captured 2026-09-26

20:23 PHT

D3 Eleven found against our own hardening build — including one our fix caused

[github.com](#) · [D-067](#)

D-066 through D-076 — eleven defects — were all raised after the 4.07 hardening merges landed on 2026-09-25, and ten of the eleven were exercised directly against that build (backend `08efeda`, frontend `5105a8b`); the eleventh was found on the deployment, which runs an older one. Finding eleven defects in the pass that was meant to close the gaps is what independent QA is for.

D-067 is the one our own disclosure fix caused — our word, not hers; she records it as a design gap whose privacy property still holds. The hardening pass stopped the backend returning the buyer's reason and the adjudicator's rejection reason to anyone without the task's read token or the operator key. That closed a real hole. It also gates on a credential that lives in backend memory and in one browser tab's session storage, and never on the **payer's wallet** — and adjudication is manual and can take a day, by which time the token is almost always gone. So the one reader the text is written for is the one reader reliably locked out, while everyone else is correctly excluded. Three previously-passing restart-drill checks broke on it. Filed as [backend #79](#); the fix she proposes is to let the payer prove themselves with the same signed challenge that opens a dispute.

D-067 — The payer loses both reasons unless they are in the tab that ran the task

- **Severity:** Major
- **Status:** Open
- **Affects:** DS-01, DS-04 (story 6.03f); DU-01, DU-02, DU-03 in the restart drill's browser half (story 6.03d, found 2026-09-26)

Also breaks the restart drill's browser half (2026-09-26, story 6.03). At backend `08efeda`, frontend `5105a8b`, real Postgres, the drill's page is a tab that never ran the task. There, `GET /api/tasks/{task}/disputes` and `GET /api/disputes/{id}` both return `reason: ''` for the payer's open dispute. So in `tools/restart-drill/browser.spec.ts`:

- DU-01 and DU-03 fail at `openAsPayer`, which waits for the payer's reason.
- DU-02 raises its dispute through the dialog after the restart and shows "Under review", then fails on its last line, the reason.
- DU-04 passes, because it proves the payer by the receipt's payer voice.
- The token-gap test still fails as expected on D-065's assertion.

Reworked the same day. The reason is now pinned in one place: a test named "DU-01 the payer's own reason is on their receipt", `test.fail()` on D-067 (0607d1a). `openAsPayer`, DU-01's post-reload check and DU-02's last line prove the payer by the receipt's own voice instead, "your wallet", which anyone else reads as "the payer's wallet" (344c55c, b77cb56, 27b0c92). DU-04 went back to the shared helper (6c82291).

Each test was then run on its own, against backend `08efeda`, frontend `5105a8b` and a real Postgres:

- DU-01 passes (18.7 min).
- DU-02 passes.
- DU-03 passes.
- DU-04 passes.
- The D-067 test fails as expected.
- The token-gap test fails as expected (D-065).

A run of the whole file in its serial order did **not** complete. DU-01's second backend boot never started within the spec's 420 s: its log is empty, so Python never finished importing. The laptop had about 190 MB available (3.8 GB total). That is the machine, not the product, but the serial run is still owed on a machine with room.

Steps to reproduce — backend `08efeda`, frontend `5105a8b`, with `tools/dispute-ui-drill/`: reject a dispute with a reason, then open the trace page with the payer's wallet connected, in a tab that does not hold the task's read token. That is any tab other than the one that ran the task, any other device, or any session after the backend has restarted.

Expected — story 6.03f: "The rejection reason is shown to the payer only", and "rejected says why".

Actual — the receipt reads "Rejected" and "The platform did not uphold this dispute: no credit was issued, Researcher's reputation is unchanged." There is no reason, and there is an empty "Your reason" label (D-068). The backend sends `reason` and `rejection_reason` only to a caller holding the task token or the operator key (`TaskReadProof.proves`, `DisputeResponse.of`). The payer's wallet is never considered. The token lives in the backend's memory, which is lost on restart and evicted after 200 tasks, and in the `sessionStorage` of the tab that ran the task. Adjudication is manual and can take up to 24 h, so by the time a verdict exists the token is almost always gone.

Impact — the one reader the reason is written for does not see it. The privacy rule itself holds: nobody else sees it either.

Resolution path — let the payer prove who they are to read the free text, for example with the same signed challenge that opens a dispute. Keep withholding it from everyone else.

Verified by — `tools/dispute-ui-drill/browser.spec.ts` "FS-07 the payer returning without the task's tab still reads why it was rejected" (`test.fail()`), pinned to D-067).

the register's entry for D-067 · <https://github.com/BIOcksmiths/Orizon-Agents-UAT-Stellar/blob/main/docs/uat/defects.md> · captured 2026-09-26 20:05 PHT

D4 The verdict: no-go[github.com](#) · [QA 6.03 evidence, §5](#) ↗

“No-go for signing off 6.03. One criterion is blocked on the deploy, four fail in code, and Deliverable 3 is not captured.” Of the story card’s eight criteria, three pass in code and none passes outright on the deployment: both on-chain artifacts are blocked behind D-050 and D-051; the window edges, the credit ceiling, the reputation cache and the rating-failure log each fail on a named open defect. Seven distinct duplicate-payment paths were attacked against the card’s minimum of four, and all seven hold at the hardening build on the Postgres store; D-058 is a separate, open path on the in-memory store.

She also fixes the order in which the switch may be turned on, and this document does not argue with it: the deployment must run backend `08efeda` or later; `MAX_REFUND_USDC` must be unset or a finite positive number; `DATABASE_URL` must be set — unconfirmed on the deployment today, since no endpoint reports the store in use (D-063). Then one live session. That list is the backbone of page 22.

5 · Recommendation

No-go for signing off 6.03. One criterion is blocked on the deploy, four fail in code, and Deliverable 3 is not captured.

Before `DISPUTE_REFUNDS_ENABLED` goes on, in this order:

1. The deploy runs backend `08efeda` or later. `/openapi.json` shows a security scheme once it does. This carries the D-053 and D-056 fixes.
2. `MAX_REFUND_USDC` on Render is unset or a finite positive number (D-054).
3. `DATABASE_URL` is still set (DU-05 checks it on every run).

Then the one live session: a settled run (needs D-039/D-050), refunds on with `API_KEY` set (D-051), an uphold through the API rather than the script (D-066), capturing the dispute tx, the refund tx and a recording. The checklist is `checklists/6.03f-phone-and-screen-reader.md`. That session turns SD-01 green and proves SD-02 on-chain. SD-03, SD-06, SD-07 and SD-08 need the fixes for D-060, D-054, D-066 and D-075.

QA’s recommendation, in her own words · <https://github.com/BI0cksmiths/Orizon-Agents-UAT-Stellar/blob/main/docs/uat/evidence/6.03-dispute-refund-rating.md> · captured 2026-09-26 20:06 PHT

Why D3 has no evidence run yet

Deliverable D3 asks for a dispute transaction and the matching partial-refund transaction **produced by the deployment**, plus a recording of the dispute UI from the same session. Everything needed to produce them is built, merged, hardened and independently tested. The run has not happened, for a reason that is not in the dispute code at all — and this page states it rather than leaving it to be discovered.

No workflow settles, so no dispute window is ever stamped. `PaymentEscrow.charge` attempts the transfer from an account that never signs it, so the charge never lands. The settlement recorder returns early when the charge produced no job id, so **no settlement record is written** — and because the dispute window is stamped on the settlement record (B6), no window ever opens and no step is ever disputable. Three merged stories of dispute, refund and rating work, and the whole trace-view dispute interface, are unreachable by any buyer on testnet. Nothing in the product says so; it looks like a feature that exists. This is QA's **D-050 (backend #67)**, and it is a consequence of the contract-level defect **D-039 (contracts #3)** rather than a bug in the dispute code. Both issues say so, and the cheapest possible tripwire is already in QA's live suite: a pinned expected failure that turns green by itself the first time a charge lands.

Blocksmiths / **Orizon-Agents-BE-Stellar** (Public)

Notifications Fork 0 Star 1

Code Issues 17 Pull requests 8 Actions Projects Security and quality Insights

No finished run is disputable: the settlement record is never written, so the dispute path is unreachable #67

New issue

rie-hash14 opened 2d ago

Severity: Critical

Reproduce

Run any workflow to completion on the deployed service, then read its disputes. From the 2026-09-24 QA re-run, `task_e534ce3b29391ae` (status: complete, spent: 6.834, both steps delivered):

```
curl -s https://orizon-xyz/api/tasks/task_e534ce3b29391ae/disputes
```

Expected

A settlement to dispute against and a closing time: `dispute_window_seconds` (default 86 400) is stamped onto the settlement record at settle time, and the trace is meant to announce dispute window open — any delivered step can be disputed until `close`.

Actual

```
{ "task_id": "task_e534ce3b29391ae", "window_close_at": null, "now": 179022636.5, "settlement": null, "disputes": [] }
```

No trace in seven runs carried a dispute-window line. `record_settlement` returns early when the charge produced no `job_id` (`api/services/execution_svc.py:1668-1682`), and the charge never lands because `PaymentEscrow.charge` cannot move the payer's funds (filed separately against the contract resp). Seven authorizations produced 0 charged events.

Impact

POST `/api/disputes`, the challenge and adjudication routes, the refund executor, the negative-rating path and the whole `/api/trace` dispute UI cannot be reached by any buyer on testnet. Nothing in the product says so: the task still reads `complete` with a non-zero `spent`.

Suggested fix

Primarily the escrow fix. Until then, consider writing a settlement row (or telling the buyer why there is none) when a charge fails, so the state is visible rather than silent.

Note

Epic 4's dispute and refund work has no QA acceptance criteria yet: UAT added EX-09 to cover reachability only.

Found in QA story 6.05 (external agent execution path), UAT defect **D-050**, re-verified on the 2026-09-24 re-run. Evidence: https://github.com/Blocksmiths/Orizon-Agents-UAT-Stellar/blob/uat/docs/uat/evidence/6_05-external-dispatch.md · log: <https://github.com/Blocksmiths/Orizon-Agents-UAT-Stellar/blob/uat/docs/uat/defects.md>

rie-hash14 2d ago Author

6.03a — still blocked (re-checked 2026-09-24 13:28 UTC)

I didn't start the session. Both blockers from this morning are unchanged, and running it now would waste the one continuous recording the story needs.

- Dispute routes are deployed. All five are in the live OpenAPI (create, challenge, read, uphold, reject), so the 2026-09-22 "zero dispute router" note is out of date.
- Refunds are switched off on the deploy (D-051 → ☐ Dispute refunds are disabled on the deployed service, so no dispute can be upheld (blocks story 6.03a) #68, Blocker). POST `/api/trace/refunds` (with optional `answers`) returns 303 `dispute_refunds_disabled` and a non-empty `API_KEY` need setting in Render. Until then, `uphold_dispute.py` would be refused even with a dispute open.
- Nothing can be disputed (D-050 → ☐ No finished run is disputable: the settlement record is never written, so the dispute path is unreachable #67, which follows from D-039 → ☐ PaymentEscrow.charge cannot move a buyer's funds: transfer is attempted from an account that never signs Orizon-Agents-Smart-Contract-Stellar's Critical). The escrow charge never lands, so no settlement is recorded. That means no receipt panel and no step to dispute.
- Tests: the dispute-path and dispute-refunds specs pass against the live stack (52 passed). DP-01 is among them because it's marked as an expected failure. It turns green by itself once a charge lands.

No wallet was signed, no recording was started and no transaction was submitted. The re-check is recorded in `record_settlement/6_03a-dispute-path.md` §7 (Blocksmiths/Orizon-Agents-UAT-Stellar · uat, 0632890).

To unblock, in order:

- Fix and redeploy the escrow (charge authorization) ☐ PaymentEscrow.charge cannot move a buyer's funds: transfer is attempted from an account that never signs Orizon-Agents-Smart-Contract-Stellar #3. DP-01 going green confirms it.
- In Render, set `DISPUTE_REFUNDS_ENABLED=true` and `API_KEY`, and confirm `DATABASE_URL`. `DATABASE_URL` can't be checked from outside.
- Book one sitting with Dan, who holds the setter key: Rie runs the buyer wallet and the recording; Dan runs the dry run, then the live uphold.

rie-hash14 1d ago Author

Story 6.03d (durability and the unconfirmed-refund path) — status 2026-09-25: the live half is blocked by this issue.

Re-checked 07:43 UTC: (BP-88) still fails as expected, so no paid run has a settlement record and there is still no dispute on the deploy to restart amount. Render restarted the service by itself during the check (`/health` uptime 9.5 s), so restarts are routine and the live test only needs a settlement to exist.

Criterion	Result
DU-01 a dispute survives a restart	Holds locally — not run live
DU-02 a settlement stays disputable after a restart	Holds locally — not run live
DU-03 an unconfirmed refund reads as pending	Holds locally — not run live

github.com · backend #67 — D-050, the dispute path unreachable · <https://github.com/Blocksmiths/Orizon-Agents-BE-Stellar/issues/67> · captured 2026-09-26 20:17 PHT

The path from here to a captured D3 evidence run is on the next page. None of it is further development on the dispute path.

Blocksmiths / **Orizon-Agents-Smart-Contract-Stellar** (Public)

Notifications Fork 0 Star 2

Code Issues 1 Pull requests 1 Actions Projects Security and quality Insights

PaymentEscrow.charge cannot move a buyer's funds: transfer is attempted from an account that never signs #3

New issue

rie-hash14 opened 2d ago

Severity: Critical

What happens

`authorize` records an `Authorize` event, takes no custody and grants no allowance. `challenge` is self-gated (`caller.require_auth()`) and ends in `send_transfer(auth.payer, agent, owner, amount)`. The SAC's transfer requires from `require_auth()` — the payer's authorization — but the charge transaction is signed by the settler alone, so the host rejects it with `Error(Auth, InvalidSignature)`.

Evidence from two QA runs against testnet

Contract `CB3PMHMG0GJL7Z2PE0QJUK6GUM97W0NDQJ3MFCY2635525F1`:

- 2026-09-17: 9 authorizations, 0 charged events.
- 2026-09-24 (after the backend redeploy; contract unchanged): 7 more authorizations, 0 charged events. The payer's balance moved only by transaction fees.

Every run still finalizes as `complete` with a `spent` figure and `charge_tx` null, and the buyer's trace carries a single `error` — `no-settlement-failed` line.

Impact

No operator has ever been paid through the escrow. The escrow's existing successful charges are the settler paying itself, which cannot stand in for a buyer paying an operator. It also makes the whole Epic 4 dispute and refund path unreachable, since a settlement record is only written when a charge lands (filed separately against the backend).

Suggested fix

Take custody of the payer's asset at `authorize` time, or have charge use `transfer:from` against an allowance the payer granted, or have the payer co-sign the charge. The deployed ABI has no `approve`, `transfer:from` or deposit endpoint, so closing this needs a new contract deployment.

Found in QA story 6.05 (external agent execution path), UAT defect **D-039**, re-verified on the 2026-09-24 re-run. Evidence: https://github.com/Blocksmiths/Orizon-Agents-UAT-Stellar/blob/uat/docs/uat/evidence/6_05-external-dispatch.md · log: <https://github.com/Blocksmiths/Orizon-Agents-UAT-Stellar/blob/uat/docs/uat/defects.md>

rie-hash14 2d ago Author

6.03a — still blocked (re-checked 2026-09-24 13:28 UTC)

I didn't start the session. Both blockers from this morning are unchanged, and running it now would waste the one continuous recording the story needs.

- Dispute routes are deployed. All five are in the live OpenAPI (create, challenge, read, uphold, reject), so the 2026-09-22 "zero dispute router" note is out of date.
- Refunds are switched off on the deploy (D-051 → ☐ Dispute refunds are disabled on the deployed service, so no dispute can be upheld (blocks story 6.03a) Orizon-Agents-BE-Stellar#68, Blocker). POST `/api/trace/refunds` (with optional `answers`) returns 303 `dispute_refunds_disabled` and a non-empty `API_KEY` need setting in Render. Until then, `uphold_dispute.py` would be refused even with a dispute open.
- Nothing can be disputed (D-050 → ☐ No finished run is disputable: the settlement record is never written, so the dispute path is unreachable Orizon-Agents-BE-Stellar#67, which follows from D-039 → ☐ PaymentEscrow.charge cannot move a buyer's funds: transfer is attempted from an account that never signs #3, Critical). The escrow charge never lands, so no settlement is recorded. That means no receipt panel and no step to dispute.
- Tests: the dispute-path and dispute-refunds specs pass against the live stack (52 passed). DP-01 is among them because it's marked as an expected failure. It turns green by itself once a charge lands.

No wallet was signed, no recording was started and no transaction was submitted. The re-check is recorded in `docs/uat/evidence/6_03a-dispute-path.md` §7 (Blocksmiths/Orizon-Agents-UAT-Stellar · uat, 0632890).

To unblock, in order:

- Fix and redeploy the escrow (charge authorization) ☐ PaymentEscrow.charge cannot move a buyer's funds: transfer is attempted from an account that never signs #3. DP-01 going green confirms it.
- In Render, set `DISPUTE_REFUNDS_ENABLED=true` and `API_KEY`, and confirm `DATABASE_URL`. `DATABASE_URL` can't be checked from outside.
- Book one sitting with Dan, who holds the setter key: Rie runs the buyer wallet and the recording; Dan runs the dry run, then the live uphold.

rie-hash14 1d ago Author

Story 6.03d (durability and the unconfirmed-refund path) — status 2026-09-25: the live half is blocked, and this issue is the root of it.

Re-checked 07:43 UTC: (BP-88) still fails as expected. The escrow charge never lands, so no paid run has a settlement record (Blocksmiths/Orizon-Agents-BE-Stellar#67), and there is no dispute on the deploy to restart amount. Render restarts the service by itself routinely (`/health` uptime was 9.5 s during the check), so the live test needs only a charge that lands.

Criterion	Result
DU-01 a dispute survives a restart	Holds locally — not run live
DU-02 a settlement stays disputable after a restart	Holds locally — not run live
DU-03 an unconfirmed refund reads as pending	Holds locally — not run live

github.com · contracts #3 — D-039, the escrow charge · <https://github.com/Blocksmiths/Orizon-Agents-Smart-Contract-Stellar/issues/3> · captured 2026-09-26 20:18 PHT

Orizon Agents — Week 3 · Technical Documentation & Demo Evidence · Stellar testnet

21 / 23

Exactly what would produce the D3 evidence run

In order, and none of it development work on the dispute path. Steps 2, 3, 4 and 6 are the ordering independent QA set out before the refund switch may be turned on; this document does not argue with it.

1 · Make a charge land	Fix the escrow so the payer's funds move (contracts #3). Without it there is nothing to dispute, whoever drives the run. This is the one true prerequisite, and it is contract work.
2 · Deploy current main	Backend <code>08efeda</code> or later, carrying the hardening pass's money-path and disclosure fixes. <code>GET /openapi.json</code> will show the <code>OperatorApiKey</code> scheme once it has — the same check C3 makes today. Render does not auto-deploy for this organisation, so this is a manual step.
3 · Set an operator key	<code>API_KEY</code> , at least 8 printable ASCII characters with no surrounding whitespace. The boot validator refuses anything shorter <i>by name</i> , rather than booting clean and then answering 401 to the operator's own key: below 8 characters the log redaction filter will not mask the value, so a shorter key prints itself into any log line that quotes it.
4 · Switch refunds on	<code>DISPUTE_REFUNDS_ENABLED=true</code> , in the same pass: setting it without a key makes the process refuse to start, deliberately, so the two go together.
5 · Confirm the database	<code>DATABASE_URL</code> must be set, or the dispute store falls back to memory and the 24-hour window becomes a promise that ends at the next restart — which a free-tier instance performs whenever it idles. No endpoint reports it, so it is checked in the dashboard.
6 · Keep the ceiling finite	Confirm <code>MAX_REFUND_USDC</code> is unset or a finite positive number: a non-finite value silently removes the refund cap (QA's D-054, still open at the hardening build).
7 · Decide the refund asset	The deployment is configured with the native XLM SAC while every amount in the interface is labelled USDC (C1). Configure a USDC SAC or correct the label — do not capture D3 evidence while the two disagree.
8 · Capture one session	A settled run; a dispute raised by the paying wallet; an uphold through the API rather than the operator script, so the running service's cached score is invalidated (QA's D-066) — producing the refund transaction and the <code>kind="dispute"</code> rating transaction, with the screen recording running continuously from before the dispute is raised until the receipt flips to "Refunded" with both Stellar Expert links.

Why that one session is enough

The receipt polls itself while a dispute is unresolved — every 30 seconds under review, every 5 while a credit is being sent, not at all once final or while the tab is hidden — so the flip from "Under review" to "Refunded" with both links happens **in the recording, without a reload**. QA reproduced exactly that in the 6.03f drill, and proved the page never reloaded by setting a marker on it before the uphold and finding it still there afterwards.

That session turns the two "not captured" rows in this document into a pair of hashes, and turns QA's first criterion green. Four of her eight criteria need code fixes beyond it, each already filed as a public issue.

Demo video: there is no Week-3 recording, and why

There is no Week-3 demo video, and the Week-3 public build post has not been published. The D3 walkthrough recording is blocked on the defect on page 21: a recording of the dispute UI is only worth making against a run that settles, and no run settles on the deployment. The tracker card for the weekly public post carries no URL, and none is invented here. Both are stated as outstanding rather than presented as met.

The most recent published demo is **Week 2's**. It is linked below and labelled as Week 2's wherever it appears in this document. It shows reputation-gated routing and the external agent execution path — **not** the dispute path this week delivered.

Week-2 build post · plays the video ·
published 2026-09-19

<https://x.com/OrizonAgents402/status/2101103657043255772>

Embed view · the same post, no login needed

<https://platform.twitter.com/embed/Tweet.html?id=2101103657043255772>



the Week-2 post, drawn by X's embed renderer ·

<https://x.com/OrizonAgents402/status/2101103657043255772> · rendered via

<https://platform.twitter.com/embed/Tweet.html?id=2101103657043255772> · captured 2026-09-26 20:02 PHT

x.com shows a blank page to a logged-out automated browser, so this capture is X's own embed view of the same post id. The Week-3 recording will be captured in the same session that produces the two deployment transactions — step 8 on page 22.